Line-by-Line Companion to `fiscal_model.py`: Every Line of Code and Its Mathematics

Abstract

This document explains *every* line of `fiscal_model.py` — the model behind the Streamlit fiscal policy app — pairing each statement with the exact mathematical object it represents, cited as **L**n for line n. One feature the app exposes as a first-class control is **productive public capital**, with production $Y_t = AK_t^{\theta_K}(K_t^G)^{\theta_G}N_t^{\theta_N}$. There is no separate "basic purchases" category (it was removed, since the utility function never values government consumption directly): total government spending splits into just public investment I^G (resource-using, accumulates into K^G) and lump-sum transfers G^T (resource-neutral). Where the code implements a nontrivial derivation (the log-linearization of the household's Euler equation and labor-leisure condition, the Blanchard–Kahn/King–Plosser–Rebelo saddle-path solution via eigendecomposition, and the *nested* fixed-point iterations needed once public capital is productive), that derivation is carried out here from the model's first-order conditions, term by term.

Contents

1 Module docstring and the model in one page (L1–L55) **1**

2 `_supply_side_impl`: the price system with public capital (L62–L101) **2**

3 `SteadyState` and its constructors (L108–L213) **5**

4 The log-linear coefficients: derivation and code (L220–L272) **9**

4.1 The static block: output as a function of $\hat{k}, \hat{k}^G, \hat{c}, \hat{g}$ (L234–L249) 9

4.2 The Euler equation coefficients (L251–L263) 11

4.3 The capital-accumulation coefficients (L265–L268) 12

5 `_kg_hat_path`: public capital's own law of motion (L275–L286) **12**

6 `_solve_saddle_path`: the Blanchard–Kahn/KPR solution (L289–L350) **13**

7 `simulate_transition`: assembling the full path (L353–L412) **15**

8 `run_experiment`: a full policy comparison (L419–L489) **18**

1 Module docstring and the model in one page (L1–L55)

The docstring (L1–L46) states the model compactly:

$$
\begin{aligned}
u_t &= \log C_t + \theta_L \log L_t, \qquad L_t = 1 - N_t && \text{(preferences)} \\
Y_t &= A\, K_t^{\theta_K} (K_t^G)^{\theta_G} N_t^{\theta_N}, \qquad \theta_K = 1 - \theta_N && \text{(technology)} \\
K_{t+1} &= (1-\delta) K_t + I_t, \qquad K_{t+1}^G = (1-\delta) K_t^G + I_t^G && \text{(capital accum.)} \\
Y_t &= C_t + I_t + I_t^G && \text{(resource constraint)} \\
\tau_t Y_t &= I_t^G + G_t^T, \qquad \tau_t = \frac{I_t^G + G_t^T}{Y_t} && \text{(government budget)}
\end{aligned}
$$

with two financing rules for a change in total government spending: both set τ_t by the *same* formula above; they differ only in whether that tax is a genuine wedge on the household's factor income (**income_tax**) or leaves it untouched (**lump_sum**, a non-distorting poll tax for the same revenue). Two solution objects are provided: an exact closed-form `steady_state` (Section 3) and a log-linearized `simulate_transition` (Sections 4–7). All quantities are *great ratios* — shares of output/the time endowment — so the model is scale-free (L43–45). There is no expectation operator anywhere in the file: the model is deterministic (perfect foresight), so every household first-order condition below is an ordinary difference equation, not a stochastic Euler equation.

```
48 from __future__ import annotations
49 from dataclasses import dataclass
50 from typing import Literal, Optional
51 import numpy as np
52 Financing = Literal["lump_sum", "income_tax"]
```

L48 Postpones type-annotation evaluation (a typing convenience only).

L50 Imports `dataclass`, used for `SteadyState`, `TransitionPath`, `PolicyExperiment`.

L51 Imports `Literal` (used at L55 to restrict the `financing` argument to exactly two string values) and `Optional` (used for `PolicyExperiment` fields that can be `None` before a path is computed).

L53 NumPy — the only numerical dependency; the saddle-path solver (Section 6) uses `numpy.linalg.eig` for a 2×2 eigendecomposition, the only linear-algebra call in the whole project.

L55 Defines `Financing` as a type restricted to the two literal strings `"lump_sum"` and `"income_tax"`.

2 `_supply_side_impl`: the price system with public capital (L62–L101)

```
62 def _supply_side_impl(theta_N: float, delta: float, r: float, A: float, tau
       : float,
63                       theta_G: float, s_IG: float, distortionary: bool, N:
                             float) -> dict:
```

L62–L63 Signature: seven primitives (five original plus θ_G, the public-capital productivity parameter, and s_{IG}, the public-investment share of output), a financing flag, and **now also** N itself — out comes a dictionary of derived prices/ratios. N is needed here (and was not, in an earlier version of this file) because of a subtlety explained in the box at the end of this section: once $\theta_G > 0$, the capital/labor ratio $\kappa = K/N$ genuinely depends on the level of N, not just on ratios.

```
64 theta_K = 1.0 - theta_N
65 tax_factor = (1.0 - tau) if distortionary else 1.0
66 if tax_factor <= 0:
67     raise ValueError("Invalid parameters: (1-tau) must be positive under
       Income Tax financing.")
68
69 kappa = ((tax_factor * theta_K * A) / (r + delta)) ** (1.0 / (1.0 - theta_K
   ))
70 alpha = A * kappa ** theta_K  # will be rescaled by KG^theta_G below
71 KG = 0.0
```

L64 $\theta_K = 1 - \theta_N$, capital's Cobb–Douglas share.

L65 The tax wedge on the household's factor income: $(1 - \tau)$ under `income_tax` financing, 1 under `lump_sum` financing. This single scalar is *the* difference between the two financing rules everywhere in the file — both set the same τ (Section 3), but only `income_tax` lets it enter as a factor-income wedge here.

L66–L67 Guards against a tax rate $\geq 100\%$ under income-tax financing.

L69 Solves the household's after-tax capital-Euler condition for the capital/labor ratio, *ignoring public capital's contribution for now* (corrected in the fixed-point loop below when $\theta_G > 0$):

$$(1 - \tau)\,\theta_K\,A\,\kappa^{\theta_K - 1} = r + \delta \tag{1}$$

(or without the $(1 - \tau)$ factor under lump-sum). Solving for κ: $\kappa = \big(\text{tax_factor} \cdot \theta_K A/(r + \delta)\big)^{1/(1-\theta_K)}$.

L70 Average product of labor *holding K^G at its $\theta_G = 0$ value*, $\alpha \equiv Y/N = A\kappa^{\theta_K}$; gets rescaled below when public capital is present and productive.

L71 Initializes $K^G = 0$, the value used unless one of the two branches below overrides it.

```
72 if theta_G > 0 and s_IG > 0:
73     # Fixed point on alpha (=Y/N): KG = s_IG*Y/delta = s_IG*alpha*N/delta.
       Working in
74     # per-N terms (kappa=K/N, KG_ppN=KG/N), dividing the technology Y=A*K^
       thetaK*KG^thetaG*N^thetaN
75     # by N gives alpha=A*kappa^thetaK*KG^thetaG*N^(thetaK+thetaG+thetaN-1)
76     # =A*kappa^thetaK*KG_ppN^thetaG*N^thetaG (thetaK+thetaN=1 cancels, but
       the thetaG
77     # exponent on N does NOT cancel -- KG is funded as a share of AGGREGATE
        output,
78     # which scales with N, so an explicit N^thetaG factor is required here)
       .
```

```
79      alpha_ppN = A * kappa ** theta_K  # Y/N ignoring KG, will multiply by
            KG_ppN^theta_G * N^theta_G
80      alpha_guess = alpha_ppN
81      N_pow = N ** theta_G
82      for _ in range(200):
83          KG_ppN = s_IG * alpha_guess / delta  # KG per unit of N
84          base = (tax_factor * theta_K * A * KG_ppN ** theta_G * N_pow) / (r
                + delta)
85          kappa = base ** (1.0 / (1.0 - theta_K))
86          alpha_new = A * kappa ** theta_K * KG_ppN ** theta_G * N_pow
87          if abs(alpha_new - alpha_guess) < 1e-12:
88              alpha_guess = alpha_new
89              break
90          alpha_guess = alpha_new
91      alpha = alpha_guess
92      KG = s_IG * alpha / delta  # KG per unit of N
93  elif s_IG > 0:
94      # theta_G=0: public capital is not productive, but it still physically
95      # accumulates from investment (KG=IG/delta at steady state) -- no fixed
96      # point needed here since KG^0=1 means it can't feed back into alpha/
            kappa.
97      KG = s_IG * alpha / delta
```

L72 Only run the (more expensive) fixed-point iteration when public capital is actually productive ($\theta_G > 0$) *and* present ($s_{IG} > 0$); when $\theta_G = 0$ (L93, L97, see the box below) the physical stock is still computed, just without any feedback into κ, α.

must be supplied. Because $\theta_K + \theta_N = 1$, dividing the technology equation $Y_t = AK_t^{\theta_K}(K_t^G)^{\theta_G}N_t^{\theta_N}$ through by N gives $\alpha \equiv Y/N = A\kappa^{\theta_K}(K^G)^{\theta_G}N^{\theta_K+\theta_G+\theta_N-1} = A\kappa^{\theta_K}(K^G)^{\theta_G}N^{\theta_G}$ — the θ_K, θ_N powers of N cancel, but the θ_G power does *not*: it survives as an explicit N^{θ_G} factor. The reason is that K^G is funded as a share of *aggregate* output ($K^G = I^G/\delta$, $I^G = s_{IG}Y$), which itself scales with N, not of output *per worker*. Writing everything per unit of N ($\kappa = K/N$, `KG_ppN` $= K^G/N$) turns the capital-Euler condition into

$$\text{tax_factor} \cdot \theta_K \, A \left(K^G/N\right)^{\theta_G} N^{\theta_G} \, \kappa^{\theta_K - 1} = r + \delta, \tag{2}$$

which still has a circular dependency (K^G/N depends on α, which depends on κ, which depends on K^G/N) — resolved by iterating on α until it stops moving, exactly as before this fix. What *changed* is the extra, non-cancelling N^{θ_G} factor: an earlier version of this function omitted it entirely (implicitly using $(K^G/N)^{\theta_G}$ alone, as if $\theta_K + \theta_G + \theta_N = 1$), which silently corrupted every steady state with $\theta_G > 0$ whenever N moved away from its calibration target. N is therefore now a required argument here, and `N_pow`$= N^{\theta_G}$ is computed once, up front, and multiplied into every α evaluation inside the loop.

L79–L90 The fixed-point sweep: L79 initializes $\alpha_{\text{ppN}} = A\kappa^{\theta_K}$ (ignoring K^G for now); L80 seeds the guess; L81 caches N^{θ_G} once (constant across iterations, so computed outside the loop); L82 up to 200 iterations; L83 recomputes K^G/N; L84 recomputes κ from (2); L85–86 recomputes $\alpha_{\text{new}} = A\kappa^{\theta_K}(K^G/N)^{\theta_G}N^{\theta_G}$; L87–90 convergence check.

L91–L92 Finalizes α and K^G (per unit of N) at their fixed-point values.

L93, L97: public capital must still accumulate at $\theta_G = 0$. An earlier version of this function set $K^G \equiv 0$ whenever $\theta_G \leq 0$, conflating "not productive" with "does not exist." But K^G's law of motion, $K^G_{t+1} = (1-\delta)K^G_t + I^G_t$, does not care whether θ_G is 0 — at a steady state $K^G = I^G/\delta$ regardless, so a positive public-investment share s_{IG} should always produce a nonzero physical capital stock, even when that stock has no effect on production. The `elif s_IG > 0` branch (L93, L97) fixes this: no fixed point is needed here (unlike the $\theta_G > 0$ branch above) because $(K^G)^0 = 1$ means K^G cannot feed back into α, κ at all when $\theta_G = 0$, so $KG = s_{IG}\alpha/\delta$ can be computed directly from the already-known α.

```
w = theta_N * alpha
q = tax_factor * theta_K * alpha / kappa
s_I = delta * kappa / alpha
return dict(theta_K=theta_K, kappa=kappa, alpha=alpha, w=w, q=q, s_I=s_I,
    KG=KG, tax_factor=tax_factor)
```

L98 The pre-tax real wage, $w = \theta_N \alpha = \theta_N Y/N$.

L99 The after-tax rental rate, $q = \text{tax_factor} \cdot \theta_K \alpha/\kappa = \text{tax_factor} \cdot \text{MPK}$ — by construction (since κ was solved from (2)), $q \equiv r + \delta$ exactly.

L100 $s_I \equiv I/Y = \delta K/Y = \delta\kappa/\alpha$.

L101 Packages eight derived quantities, including `tax_factor` itself (reused by every caller) and `KG` (public capital per unit of N).

3 `SteadyState` and its constructors (L108–L213)

```
@dataclass
class SteadyState:
    theta_N: float
    theta_K: float
    theta_G: float
    delta: float
    r: float
    A: float
    tau: float
    theta_L: float
    s_G: float        # total government spending share (IG+G_T)/Y
    s_IG: float
    s_GT: float
    distortionary: bool
    kappa: float
    alpha: float
    w: float
    q: float
    s_I: float
    s_C: float
    N: float
    L: float
    Y: float
```

```
    K: float
    KG: float
    C: float
    I: float
    G: float          # total government spending IG+G_T
    IG: float
    GT: float

    def as_dict(self) -> dict:
        return self.__dict__.copy()
```

L108–L139 A plain dataclass holding every parameter *and* every derived great ratio and level at one steady state: six primitives, the calibrated θ_L, the two-way composition (s_G, s_{IG}, s_{GT}) and the financing flag, the seven `_supply_side_impl` outputs $(\kappa, \alpha, w, q, s_I, s_C$ — s_C is added here, not returned by that function itself), and the levels $(N, L, Y, K, K^G, C, I, G, I^G, G^T)$ — twenty-eight fields in total.

L139–L140 `as_dict` returns a shallow copy of the attribute dictionary — used by `test_model.py` to print every field by name.

```
def calibrate_theta_L(theta_N: float, delta: float, r: float, A: float,
   theta_G: float,
                      s_IG: float, s_GT: float, distortionary: bool,
                      N_target: float) -> float:
    tau = s_IG + s_GT
    N = N_target
    supply = _supply_side_impl(theta_N, delta, r, A, tau, theta_G, s_IG,
       distortionary, N)
    s_C = 1.0 - supply["s_I"] - s_IG
    if s_C <= 0:
        raise ValueError("Government share too large: steady-state
           consumption share <= 0.")
    theta_L = supply["tax_factor"] * theta_N * (1.0 - N) / (s_C * N)
    if theta_L <= 0:
        raise ValueError("Implied theta_L <= 0; adjust N_target or the tax/
           spending settings.")
    return theta_L
```

L143–L145 Signature: given the technology/policy primitives and a target labor supply $\bar{N} \equiv$`N_target`, returns the leisure-preference weight θ_L that makes steady-state labor supply equal $\bar{N}$.

L148 $\tau = s_{IG} + s_{GT}$ — the government-budget identity, holding regardless of financing rule.

L149 $N =$`N_target`: at the calibration point, by definition, N *is* exactly the target, so it can be passed straight into `_supply_side_impl` (Section 2) rather than solved for. This is what makes the N^{θ_G} fix of Section 2 exact here, with no outer iteration needed — unlike `steady_state` below, where N is *not* known in advance.

L150 Calls Section 2's function.

L151 $s_C = 1 - s_I - s_{IG}$ — note only s_{IG} (not s_{GT}) reduces the resource constraint, since transfers are rebated to households rather than spent on goods.

L152–L153 Guards against $s_C \leq 0$.

L154 The calibration formula, from the labor-leisure FOC

$$\theta_L \frac{C}{L} = \text{tax_factor} \cdot w \tag{3}$$

solved for θ_L using $w = \theta_N \alpha$, $C = s_C Y$, $\alpha = Y/N$, $L = 1 - N$:

$$\theta_L = \frac{\text{tax_factor} \cdot \theta_N (1 - N)}{s_C N}. \tag{4}$$

L155–L156 Guards against a non-positive θ_L.

```
160 def steady_state(theta_N: float, delta: float, r: float, A: float, theta_G:
        float,
161                  s_IG: float, s_GT: float, distortionary: bool,
162                  theta_L: float) -> SteadyState:
163     tau = s_IG + s_GT
164     s_G = tau
165     N = 0.3
166     supply = None
167     for _ in range(200):
168         supply = _supply_side_impl(theta_N, delta, r, A, tau, theta_G, s_IG
                , distortionary, N)
169         s_I = supply["s_I"]
170         s_C = 1.0 - s_I - s_IG
171         if s_C <= 0:
172             raise ValueError("Government share too large: steady-state
                    consumption share <= 0.")
173         tax_factor = supply["tax_factor"]
174         N_new = tax_factor * theta_N / (theta_L * s_C + tax_factor *
                theta_N)
175         if not (0 < N_new < 1):
176             raise ValueError(f"Implied labor supply N={N_new:.3f} is
                    outside (0,1).")
177         if abs(N_new - N) < 1e-12:
178             N = N_new
179             break
180         N = N_new
181     L = 1.0 - N
```

L160–L162 Signature: same primitives plus θ_L (now given, not calibrated), returning a full `SteadyState`.

L165–L166 $\tau = s_{IG} + s_{GT}$, $s_G = \tau$.

L167–L186: why `steady_state` needs an *outer* fixed point on N, on top of `_supply_side_impl`'s own inner one. Unlike `calibrate_theta_L`, this function is *not* told N in advance — N is one of its outputs, determined by the labor-leisure condition (3). But when $\theta_G > 0$ and $s_{IG} > 0$, Section 2 showed that `_supply_side_impl` itself needs N as an *input* (through the N^{θ_G} term). This is circular at one level higher than before: N depends on s_C, which depends on the supply side, which now depends on N. The fix is the same idea applied one level up — iterate: guess N (L171, starting from an arbitrary $N = 0.3$), compute the supply side at that guess (L174), de-

rive the labor-leisure implied N_{new} (L180), and repeat until it stops moving (L183–185). When $\theta_G = 0$ or $s_{IG} = 0$, the supply side does not actually depend on N at all (Section 2), so this loop converges after exactly one pass, with no behavior change from before this fix was added.

L171 Initial guess $N = 0.3$ (arbitrary; only matters for iteration count, not the fixed point reached).

L173–L185 The outer loop, up to 200 iterations.

L174 Recomputes the supply side at the current guess for N.

L175–L178 s_I from the supply side; $s_C = 1 - s_I - s_{IG}$; guards against $s_C \leq 0$.

L180 Solves (3) for a new N: from $\theta_L s_C N = \text{tax_factor} \cdot \theta_N (1 - N)$,

$$N = \frac{\text{tax_factor} \cdot \theta_N}{\theta_L s_C + \text{tax_factor} \cdot \theta_N}. \tag{5}$$

The algebraic inverse of L154 — plugging L154's θ_L back in returns exactly $N =$`N_target` once the loop converges.

L181–L182 Guards against infeasible labor supply.

L183–L186 Convergence check and update.

L187 $L = 1 - N$, once the loop has converged.

```
189 Y = supply["alpha"] * N
190 K = supply["kappa"] * N
191 KG = supply["KG"] * N
192 C = s_C * Y
193 I = s_I * Y
194 IG = s_IG * Y
195 GT = s_GT * Y
196 G = IG + GT
```

L189–L191 $Y = \alpha N$, $K = \kappa N$, and $K^G = (K^G/N) \cdot N$ — converting the per-N ratios from Section 2 into true levels now that the fixed-point-converged N is known.

L192–L193 $C = s_C Y$, $I = s_I Y$.

L194–L195 $I^G = s_{IG} Y$, $G^T = s_{GT} Y$.

L196 $G = I^G + G^T$, total government spending; by construction $C + I + I^G = (s_C + s_I + s_{IG})Y = Y$ exactly.

```
198 return SteadyState(
199     theta_N=theta_N, theta_K=supply["theta_K"], theta_G=theta_G, delta=
        delta, r=r, A=A,
200     tau=tau, theta_L=theta_L, s_G=s_G, s_IG=s_IG, s_GT=s_GT,
201     distortionary=distortionary, kappa=supply["kappa"], alpha=supply["alpha
        "],
202     w=supply["w"], q=supply["q"], s_I=s_I, s_C=s_C, N=N, L=L, Y=Y, K=K, KG=
        KG, C=C, I=I,
203     G=G, IG=IG, GT=GT,
204 )
```

L198–L204 Assembles all twenty-eight fields into the returned `SteadyState`.

```
207 def steady_state_for_policy(theta_N: float, delta: float, r: float, A:
        float, theta_G: float,
208                                s_G_total: float, f_IG: float, f_GT: float,
209                                theta_L: float, distortionary: bool) ->
                                       SteadyState:
210     s_IG, s_GT = s_G_total * f_IG, s_G_total * f_GT
211     return steady_state(theta_N, delta, r, A, theta_G, s_IG, s_GT,
            distortionary, theta_L)
```

L207–L209 Signature: takes a *total* government-spending ratio $s_{G,\text{total}}$ and fixed composition fractions f_{IG}, f_{GT} (summing to 1), rather than the two composition levels directly. This is the function the app calls once for the baseline policy and once for the counterfactual, holding the composition fractions fixed across the two.

L212 Splits the total by the given fractions.

L213 Calls Section 3's `steady_state`.

source constraint. The government budget is $\tau_t Y_t = I_t^G + G_t^T$. The household budget differs by financing rule:

$$\text{income_tax:}\quad C_t + I_t = (1-\tau_t)(w_t N_t + r_t K_t) + G_t^T \qquad \text{lump_sum:}\quad C_t + I_t = w_t N_t + r_t K_t - \tau_t Y_t + G_t^T. \tag{6}$$

Combined with $w_t N_t + r_t K_t = Y_t$ and the government budget identity, both cases reduce *in the aggregate* to $Y_t = C_t + I_t + I_t^G$ — transfers are received by the household but exactly offset by the tax it pays, so they wash out of the resource constraint even though they matter for household-level after-tax income.

4 The log-linear coefficients: derivation and code (L220–L272)

The code's own docstring (L221–L226) states the four blocks to be solved:

$$\hat{y}_t = y_k \hat{k}_t + y_{kg} \hat{k}_t^G + y_c \hat{c}_t + y_g \hat{g}_t \qquad \text{(static block)} \tag{7}$$

$$\hat{k}_{t+1} = \Phi_{kk} \hat{k}_t + \Phi_{kc} \hat{c}_t + \Phi_{kg} \hat{g}_t + \Phi_{k,kg} \hat{k}_t^G \qquad \text{(capital accumulation)} \tag{8}$$

$$\hat{c}_t = \text{coef}_c \hat{c}_{t+1} + \text{coef}_k \hat{k}_{t+1} + \text{coef}_g \hat{g}_{t+1} + \text{coef}_{kg} \hat{k}_{t+1}^G \qquad \text{(Euler equation)} \tag{9}$$

where a hat denotes a log-deviation from the steady state `ss`. This section derives all twelve coefficients from the production function, the labor-leisure FOC (3), and the household's capital-Euler equation.

```
220 def _log_linear_coeffs(ss: SteadyState) -> dict:
```

L220 Signature: takes only the steady state to linearize around; the financing flag lives on `ss.distortionary`.

```
228 theta_N, theta_K, theta_G = ss.theta_N, ss.theta_K, ss.theta_G
229 L = ss.L
230 tau = ss.tau
231 s_C, s_I, s_IG = ss.s_C, ss.s_I, ss.s_IG
232 delta, r = ss.delta, ss.r
```

L228–L232 Unpacks every steady-state quantity needed below, including s_{IG} (the resource-using share; s_{GT} is not needed here, since transfers never enter the resource-constraint-based formulas that follow).

4.1 The static block: output as a function of $\hat{k}, \hat{k}^G, \hat{c}, \hat{g}$ (L234–L249)

Derivation. Log-linearize the production function:

$$\hat{y}_t = \theta_K \hat{k}_t + \theta_G \hat{k}_t^G + \theta_N \hat{n}_t. \tag{10}$$

Log-linearize the labor-leisure FOC (3): writing it as $\theta_L C_t = \text{tax_factor}_t \theta_N Y_t (1 - N_t)/N_t$ and differentiating, using $d\log(1 - N_t) = -\frac{N}{L}\hat{n}_t$,

$$\hat{c}_t = \widehat{(\text{tf})}_t + \hat{y}_t - \frac{\hat{n}_t}{L}. \tag{11}$$

Case `lump_sum` (tax_factor$\equiv 1$, $\widehat{(\text{tf})}_t \equiv 0$). Solving (11) for $\hat{n}_t$: $\hat{n}_t = L(\hat{y}_t - \hat{c}_t)$. Substituting into (10):

$$\hat{y}_t \underbrace{(1 - \theta_N L)}_{D} = \theta_K \hat{k}_t + \theta_G \hat{k}_t^G - \theta_N L\, \hat{c}_t, \tag{12}$$

$$\Longrightarrow \quad y_k = \frac{\theta_K}{D}, \quad y_{kg} = \frac{\theta_G}{D}, \quad y_c = -\frac{\theta_N L}{D}, \quad y_g = 0. \tag{13}$$

```
234 D = 1.0 - theta_N * L
235 if ss.distortionary:
236     # See derivation note: y_hat*(D - theta_N*L*tau/(1-tau)) = theta_K*
        k_hat +
237     # theta_G*kg_hat - theta_N*L*c_hat - theta_N*L*tau/(1-tau)*g_hat.
238     Dp = D - theta_N * L * tau / (1.0 - tau)
239     y_k = theta_K / Dp
240     y_kg = theta_G / Dp
241     y_c = -theta_N * L / Dp
242     y_g = -theta_N * L * tau / ((1.0 - tau) * Dp)
243     tax_wedge = tau / (1.0 - tau)
244 else:
245     y_k = theta_K / D
246     y_kg = theta_G / D
247     y_c = -theta_N * L / D
248     y_g = 0.0
249     tax_wedge = 0.0
```

L234 $D \equiv 1 - \theta_N L$.

L245–L249 Exactly implements (13): no $\hat{g}_t$ term because g never enters the lump-sum labor-leisure FOC, and $y_{kg} = \theta_G/D$ shares y_k's denominator since $\hat{k}_t^G$ entered (10) exactly like $\hat{k}_t$.

Case `income_tax`. Differentiating $\tau_t Y_t = G_t$ (total government spending) gives

$$\widehat{(\text{tf})}_t = \frac{\tau}{1 - \tau}(\hat{y}_t - \hat{g}_t). \tag{14}$$

Substituting into (11) and solving for $\hat{n}_t$:

$$\hat{n}_t = L\left[\frac{\hat{y}_t}{1-\tau} - \hat{c}_t - \frac{\tau}{1-\tau}\hat{g}_t\right] \tag{15}$$

— exactly the formula `simulate_transition` uses directly (Section 7, L389–L392). Substituting back into (10) and collecting $\hat{y}_t$ terms:

$$\hat{y}_t \left(\underbrace{D - \frac{\theta_N L\tau}{1-\tau}}_{D_p}\right) = \theta_K \hat{k}_t + \theta_G \hat{k}_t^G - \theta_N L\hat{c}_t - \frac{\theta_N L\tau}{1-\tau}\hat{g}_t. \tag{16}$$

L238 $D_p \equiv D - \theta_N L\tau/(1-\tau)$.

L239–L242 $y_k = \theta_K/D_p$, $y_{kg} = \theta_G/D_p$, $y_c = -\theta_N L/D_p$, $y_g = -\theta_N L\tau/[(1-\tau)D_p]$ — matching (16).

L243 tax_wedge $\equiv \tau/(1-\tau)$, reused in the Euler-equation coefficients (Section 4.2).

4.2 The Euler equation coefficients (L251–L263)

Derivation. The after-tax capital-Euler equation is, under perfect foresight (no expectation operator, since this model has no stochastic element),

$$\frac{1}{C_t} = \beta\,\frac{1 + \mathrm{tf}_{t+1}\,R_{t+1} - \delta}{C_{t+1}}, \qquad R_{t+1} \equiv \theta_K \frac{Y_{t+1}}{K_{t+1}}. \tag{17}$$

At steady state $1/\beta = \mathrm{tf}\cdot R + 1 - \delta = q + 1 - \delta$, and $q \equiv r + \delta$, so $\beta = 1/(1+r)$. Writing $X_{t+1} \equiv \mathrm{tf}_{t+1}R_{t+1} + 1 - \delta$ and log-linearizing $1/C_t = \beta X_{t+1}/C_{t+1}$:

$$\hat{c}_t = \hat{c}_{t+1} - \hat{X}_{t+1}, \qquad \hat{X}_{t+1} = \underbrace{\beta q}_{\Omega}\left[\widehat{(\mathrm{tf})}_{t+1} + \hat{R}_{t+1}\right]. \tag{18}$$

Since $R_t = \theta_K Y_t/K_t$, $\hat{R}_t = \hat{y}_t - \hat{k}_t$; using the static block, $\hat{y}_t - \hat{k}_t = (y_k - 1)\hat{k}_t + y_{kg}\hat{k}_t^G + y_c\hat{c}_t + y_g\hat{g}_t$. Substituting this and (14) (or 0, lump-sum), then $\hat{y}_{t+1}$'s own static-block expression where needed, and collecting coefficients on $\hat{k}_{t+1}, \hat{k}_{t+1}^G, \hat{c}_{t+1}, \hat{g}_{t+1}$ in $\hat{c}_t = \hat{c}_{t+1} - \hat{X}_{t+1}$, gives the four coefficients below.

```
251 beta = 1.0 / (1.0 + r)
252 q = r + delta
253 Omega = beta * q  # = q/(1+r)
254
255 ymk_k = y_k - 1.0
256 ymk_c = y_c
257 ymk_g = y_g
258 ymk_kg = y_kg
259
260 coef_c = 1.0 - Omega * ymk_c - Omega * tax_wedge * y_c
261 coef_k = -Omega * ymk_k - Omega * tax_wedge * y_k
262 coef_g = -Omega * ymk_g + Omega * tax_wedge * (1.0 - y_g)
263 coef_kg = -Omega * ymk_kg - Omega * tax_wedge * y_kg
```

L251 $\beta = 1/(1+r)$.

L252 $q = r + \delta$.

L253 $\Omega \equiv \beta q$.

L255–L258 $\hat{R}_t = \hat{y}_t - \hat{k}_t$'s coefficients: `ymk_k`$= y_k - 1$, `ymk_c`$= y_c$, `ymk_g`$= y_g$, `ymk_kg`$= y_{kg}$.

L260 $\text{coef}_c = 1 - \Omega \cdot \text{ymk_c} - \Omega \cdot \text{tax_wedge} \cdot y_c$.

L261 $\text{coef}_k = -\Omega \cdot \text{ymk_k} - \Omega \cdot \text{tax_wedge} \cdot y_k$.

L262 $\text{coef}_g = -\Omega \cdot \text{ymk_g} + \Omega \cdot \text{tax_wedge} \cdot (1 - y_g)$.

L263 $\text{coef}_{kg} = -\Omega \cdot \text{ymk_kg} - \Omega \cdot \text{tax_wedge} \cdot y_{kg}$ — structurally identical to `coef_k`'s template (both $\hat{k}_{t+1}$ and $\hat{k}^G_{t+1}$ affect the Euler equation only through $\hat{R}_{t+1} = \hat{y}_{t+1} - \hat{k}_{t+1}$), minus the "$-1$" term since $\hat{k}^G_{t+1}$ never appears *subtracted* the way $\hat{k}_{t+1}$ does.

4.3 The capital-accumulation coefficients (L265–L268)

Derivation. Log-linearizing $K_{t+1} = (1-\delta)K_t + I_t$ (using $I = \delta K$ at steady state) and the resource constraint $Y_t = C_t + I_t + I^G_t$ (using $s_C + s_I + s_{IG} = 1$):

$$\hat{\imath}_t = \frac{y_k}{s_I}\hat{k}_t + \frac{y_{kg}}{s_I}\hat{k}^G_t + \frac{y_c - s_C}{s_I}\hat{c}_t + \frac{y_g - s_{IG}}{s_I}\hat{g}_t, \tag{19}$$

substituted into $\hat{k}_{t+1} = (1-\delta)\hat{k}_t + \delta\hat{\imath}_t$:

```
265 Phi_kk = (1.0 - delta) + delta * y_k / s_I
266 Phi_kc = delta * (y_c - s_C) / s_I
267 Phi_kg = delta * (y_g - s_IG) / s_I
268 Phi_k_kg = delta * y_kg / s_I
```

L265 $\Phi_{kk} = (1-\delta) + \delta y_k / s_I$.

L266 $\Phi_{kc} = \delta(y_c - s_C)/s_I$.

L267 $\Phi_{kg} = \delta(y_g - s_{IG})/s_I$ — s_{IG}, not s_G: only resource-using spending crowds out investment.

L268 $\Phi_{k,kg} = \delta y_{kg}/s_I$ — public capital affects next period's private capital only through its effect on $\hat{y}_t$, with no direct resource-crowding term (it's a stock, not a competing flow).

```
270 return dict(y_k=y_k, y_kg=y_kg, y_c=y_c, y_g=y_g, s_GR=s_IG,
271             coef_c=coef_c, coef_k=coef_k, coef_g=coef_g, coef_kg=coef_kg,
272             Phi_kk=Phi_kk, Phi_kc=Phi_kc, Phi_kg=Phi_kg, Phi_k_kg=Phi_k_kg)
```

L270–L271 Returns all twelve coefficients plus s_{IG} (keyed `"s_GR"` for historical reasons, predating the removal of the separate "basic purchases" category, when the resource-using share was $s_{GB} + s_{IG}$ rather than just s_{IG}).

5 `_kg_hat_path`: public capital's own law of motion (L275–L286)

Derivation. Log-linearizing $K^G_{t+1} = (1-\delta)K^G_t + I^G_t$ (using $I^G = \delta K^G$ at steady state), and noting that public investment moves proportionally with total government spending at fixed composition fractions ($I^G_t = f_{IG} G_t \Rightarrow \hat{\imath}^G_t = \hat{g}_t$):

$$\hat{k}^G_{t+1} = (1-\delta)\hat{k}^G_t + \delta\, \hat{g}_t. \tag{20}$$

Public capital's law of motion does not depend on $\hat{k}_t$ or $\hat{c}_t$ at all — it is driven exogenously by the (known) government-spending path — which is why the model does not need a 3×3 generalization of the Blanchard–Kahn eigendecomposition (Section 6 stays 2×2).

```
275 def _kg_hat_path(delta: float, g_full: np.ndarray, kg0: float) -> np.
    ndarray:
276     T = len(g_full)
277     kg = np.zeros(T + 1)
278     kg[0] = kg0
279     g_ext = np.concatenate([g_full, [0.0]])
280     for t in range(T):
281         kg[t + 1] = (1.0 - delta) * kg[t] + delta * g_ext[t]
282     return kg
```

L275 Signature: returns $T+1$ values, $\hat{k}^G_0, \ldots, \hat{k}^G_T$ — one more than the input length, so the Euler-equation forcing at the last simulated period $t = T-1$ (needing $\hat{k}^G_T$) is always available.

L280–L281 T, the horizon; allocates the length-$(T+1)$ output array.

L282 Seeds $\hat{k}^G_0 =$`kg0`.

L283 Appends a trailing zero (shock has died out by the horizon's end).

L284–L285 Implements (20) by simple forward iteration — safe here (a stable, first-order, one-variable recursion with root $1-\delta \in (0,1)$).

6 `_solve_saddle_path`: the Blanchard–Kahn/KPR solution (L289–L350)

```
289 def _solve_saddle_path(Phi_kk: float, Phi_kc: float, Phi_kg: float,
    Phi_k_kg: float,
290                       coef_c: float, coef_k: float, coef_g: float,
                              coef_kg: float,
291                       k0: float, g_full: np.ndarray, kg_full: np.ndarray)
                               -> tuple[np.ndarray, np.ndarray]:
```

L289–L291 Signature: the eight coefficients from Section 4 involving $\hat{k}$ or $\hat{c}$, an initial capital log-deviation k_0, `g_full`, and the length-$(T+1)$ public-capital path `kg_full` from Section 5, supplied as a *known* forcing input. The function's own docstring calls this a "linear perfect-foresight system" — there is no expectation operator anywhere, since the model has no stochastic element; every "future" value referenced below is a known quantity along the announced policy path, not a rational expectation over uncertain outcomes.

Setting up a first-order forward system. Solving (9) for $\hat{c}_{t+1}$ using (8)'s expression for $\hat{k}_{t+1}$ and substituting gives:

$$\begin{pmatrix} \hat{k}_{t+1} \\ \hat{c}_{t+1} \end{pmatrix} = \underbrace{\begin{pmatrix} \Phi_{kk} & \Phi_{kc} \\ -\dfrac{\text{coef}_k \Phi_{kk}}{\text{coef}_c} & \dfrac{1 - \text{coef}_k \Phi_{kc}}{\text{coef}_c} \end{pmatrix}}_{M} \begin{pmatrix} \hat{k}_t \\ \hat{c}_t \end{pmatrix} + \begin{pmatrix} \Phi_{kg} \hat{g}_t + \Phi_{k,kg} \hat{k}_t^G \\ -\dfrac{\text{coef}_k (\Phi_{kg} \hat{g}_t + \Phi_{k,kg} \hat{k}_t^G)}{\text{coef}_c} - \dfrac{\text{coef}_g}{\text{coef}_c} \hat{g}_{t+1} - \dfrac{\text{coef}_{kg}}{\text{coef}_c} \hat{k}_{t+1}^G \end{pmatrix}. \tag{21}$$

M itself is unaffected by public capital — built only from $\Phi_{kk}, \Phi_{kc}, \text{coef}_k, \text{coef}_c$; public capital enters purely through the additive forcing vector.

```
311 M = np.array([[Phi_kk, Phi_kc],
312                [-coef_k * Phi_kk / coef_c, (1.0 - coef_k * Phi_kc) / coef_c
                       ]])
```

L311–L312 Builds M exactly as in (21).

Diagonalizing M. $M = V \,\text{diag}(\lambda_s, \lambda_u) V^{-1}$; saddle-path stability requires $|\lambda_s| < 1 < |\lambda_u|$ — exactly one root inside the unit circle (for the one predetermined variable $\hat{k}_t$) and one outside (for the one jump variable $\hat{c}_t$). At sufficiently extreme calibrations (very high income-tax rates combined with a low investment share) M can fail this condition entirely, with *both* eigenvalues landing outside the unit circle — a genuine breakdown of the linearized dynamics, not a numerical bug; the app catches this and falls back to reporting the two exact steady states without a transition path.

```
313 vals, vecs = np.linalg.eig(M)
314 if np.max(np.abs(vals.imag)) > 1e-8:
315     raise RuntimeError("Complex eigenvalues encountered; parameters imply
            oscillatory "
316                        "dynamics not supported by this simplified solver."
                           )
317 vals = vals.real
318 vecs = vecs.real
319 stable_idx = int(np.argmin(np.abs(vals)))
320 unstable_idx = 1 - stable_idx
321 lam_s, lam_u = vals[stable_idx], vals[unstable_idx]
322 if not (abs(lam_s) < 1.0 < abs(lam_u)):
323     raise RuntimeError("Model is not saddle-path stable for these
            parameters "
324                        "(need exactly one root inside and one outside the
                           unit circle).")
325 V = vecs[:, [stable_idx, unstable_idx]]
326 Vinv = np.linalg.inv(V)
```

L313 NumPy's eigenvalue solver.

L314–L317 Guards against complex eigenvalues; discards negligible imaginary parts.

L319 Stable eigenvalue = smaller absolute value.

L320 The other index is unstable.

L321 Reads off λ_s, λ_u.

L322–L324 Verifies true saddle-path stability; raises if not (see the discussion above).

L325 Reorders V's columns (stable, unstable).

L326 Inverts V, needed to rotate $(\hat{k}_t, \hat{c}_t)$ into the eigenbasis where the system decouples.

```
T = len(g_full)
g_ext = np.concatenate([g_full, [0.0]])
kg_now = kg_full[:T]      # kg_hat_t for t=0..T-1
kg_next = kg_full[1:T + 1]  # kg_hat_{t+1} for t=0..T-1
Nk = Phi_kg * g_ext[:T] + Phi_k_kg * kg_now
Nc = (-coef_k * (Phi_kg * g_ext[:T] + Phi_k_kg * kg_now) - coef_g * g_ext
    [1:T + 1]
      - coef_kg * kg_next) / coef_c
N = np.stack([Nk, Nc], axis=1)
n = N @ Vinv.T  # forcing rotated into the eigen-basis: n[:,0]=stable comp
    ., n[:,1]=unstable
```

L328 T, the simulation horizon.

L329 Trailing zero appended to `g_full`.

L330–L331 Slices the length-$(T+1)$ `kg_full` into "now" and "next" views.

L332 $N_k[t] = \Phi_{kg}\hat{g}_t + \Phi_{k,kg}\hat{k}^G_t$.

L333–L334 $N_c[t]$, using both contemporaneous and one-period-ahead forcing, per (21).

L335 Stacks N_k, N_c.

L336 Rotates into the eigenbasis.

```
zeta2 = np.zeros(T)
running = 0.0
for t in range(T - 1, -1, -1):
    running = (n[t, 1] + running) / lam_u
    zeta2[t] = -running
```

L338–L342 Solves the unstable (jump) component *backward* (a naive forward iteration would blow up since $|\lambda_u| > 1$), giving the discounted present value of all remaining forcing: $\zeta_{2,t} = -\sum_{j=0}^{T-1-t} n_{t+j,1}/\lambda_u^{j+1}$. This is also exactly why a "temporary" shock is only well-posed once the *true, long-run reference point* of the linearization is identified correctly: if the forcing `n[:,1]` never truly settles back to a value consistent with `ss_ref` by the end of the horizon (e.g. a shock modeled as reverting to some other, non-`ss_ref` level forever), this backward recursion's implicit "no more forcing beyond T" assumption is violated and the resulting path can diverge instead of converging.

```
    zeta1 = np.zeros(T)
    zeta1[0] = (k0 - V[0, 1] * zeta2[0]) / V[0, 0]
    for t in range(T - 1):
        zeta1[t + 1] = lam_s * zeta1[t] + n[t, 0]

    z = V @ np.vstack([zeta1, zeta2])
    return z[0, :], z[1, :]
```

L344–L345 Pins $\zeta_{1,0} = (k_0 - V_{01}\zeta_{2,0})/V_{00}$ from the true initial condition $\hat{k}_0 = k_0$.

L346–L347 Propagates $\zeta_{1,t}$ forward (safe since $|\lambda_s| < 1$).

L349 Rotates back: $\begin{pmatrix} \hat{k}_t \\ \hat{c}_t \end{pmatrix} = V\zeta_t$.

L350 Returns $\{\hat{k}_t\}, \{\hat{c}_t\}$.

7 `simulate_transition`: assembling the full path (L353–L412)

```
@dataclass
class TransitionPath:
    years: np.ndarray
    Y: np.ndarray  # % deviation from the steady state being linearized
        around
    C: np.ndarray
    I: np.ndarray
    K: np.ndarray
    KG: np.ndarray
    N: np.ndarray
    W: np.ndarray
    G: np.ndarray
    r_bp: np.ndarray  # real interest rate, deviation in basis points (
        annualized)
```

L353–L364 Holds one array per series shown in the app's transition-dynamics charts, all in percent deviations from a reference steady state.

```
def simulate_transition(ss_ref: SteadyState, g_path: np.ndarray,
                        k0: float = 0.0, kg0: float = 0.0, T_sim: int =
                            300) -> TransitionPath:
    coeffs = _log_linear_coeffs(ss_ref)
    y_k, y_kg, y_c, y_g, s_GR = coeffs["y_k"], coeffs["y_kg"], coeffs["y_c"
        ], coeffs["y_g"], coeffs["s_GR"]

    g_full = np.zeros(T_sim)
    n_g = min(len(g_path), T_sim)
    g_full[:n_g] = g_path[:n_g]

    L = ss_ref.L
    tau = ss_ref.tau
```

L367–L368 Signature: steady state to linearize around, government-spending shock path, initial capital log-deviations (k_0, kg_0, default 0), simulation horizon.

L372–L373 Computes and unpacks the coefficients.

L375–L377 Zero-pads `g_path` to length T_{sim}.

L379–L380 Unpacks L, τ.

```
382 kg_full_ext = _kg_hat_path(ss_ref.delta, g_full, kg0)  # length T_sim+1
383 k, c = _solve_saddle_path(coeffs["Phi_kk"], coeffs["Phi_kc"], coeffs["
    Phi_kg"], coeffs["Phi_k_kg"],
384                          coeffs["coef_c"], coeffs["coef_k"], coeffs["
                                coef_g"], coeffs["coef_kg"],
385                          k0, g_full, kg_full_ext)
386 kg_full = kg_full_ext[:T_sim]
387 y = y_k * k + y_kg * kg_full + y_c * c + y_g * g_full
```

L382 Solves public capital's path first (it does not depend on k, c).

L383–L385 Calls the saddle-path solver.

L386 Trims the extra period.

L387 $\hat{y}_t = y_k \hat{k}_t + y_{kg} \hat{k}^G_t + y_c \hat{c}_t + y_g \hat{g}_t$, the static block (7).

```
389 if ss_ref.distortionary:
390     n = L * (y - c) - (L * tau / (1.0 - tau)) * (g_full - y)
391 else:
392     n = L * (y - c)
393 w = y - n  # wage = MPL, log-deviation
394
395 i = (y - ss_ref.s_C * c - s_GR * g_full) / ss_ref.s_I
```

L389–L392 Implements (15) or its lump-sum simplification directly.

L393 $\hat{w}_t = \hat{y}_t - \hat{n}_t$.

L395 $\hat{\imath}_t = (\hat{y}_t - s_C \hat{c}_t - s_{IG} \hat{g}_t)/s_I$, from Section 4.3.

```
397 c_pct = 100.0 * c
398 y_pct = 100.0 * y
399 i_pct = 100.0 * i
400 k_pct = 100.0 * k
401 kg_pct = 100.0 * kg_full
402 n_pct = 100.0 * n
403 w_pct = 100.0 * w
404 g_pct = 100.0 * g_full
405
406 r_dev_bp = np.zeros_like(c)
407 r_dev_bp[:-1] = 10000.0 * (1.0 + ss_ref.r) * (c[1:] - c[:-1])
408 r_dev_bp[-1] = r_dev_bp[-2]
```

```
409
410 years = np.arange(T_sim)
411 return TransitionPath(years=years, Y=y_pct, C=c_pct, I=i_pct, K=k_pct, KG=
    kg_pct, N=n_pct,
412                       W=w_pct, G=g_pct, r_bp=r_dev_bp)
```

L397–L404 Converts every log-deviation to percent.

L406–L408 Real interest rate deviation in basis points, via a forward difference of $\{\hat{c}_t\}$ ($dr_{t+1} \approx (1 + r)(\hat{c}_{t+1} - \hat{c}_t)$); the last period repeats the second-to-last value.

L410–L412 Assembles and returns the `TransitionPath`.

8 `run_experiment`: a full policy comparison (L419–L489)

```
419 @dataclass
420 class PolicyExperiment:
421     ss_old: SteadyState
422     ss_new: SteadyState
423     path: Optional[TransitionPath]
424     multiplier_long_run: float
425     multiplier_impact: Optional[float]
```

L419–L425 Bundles the before/after steady states, the transition path, and the two headline multipliers.

```
428 def run_experiment(theta_N: float, delta: float, r: float, A: float,
    theta_G: float,
429                    s_G_old: float, s_G_new: float, f_IG: float, f_GT:
                           float,
430                    N_target: float, financing: Financing,
431                    permanent: bool, duration_years: int = 4, T_sim: int =
                           300) -> PolicyExperiment:
432     distortionary = financing == "income_tax"
433     s_IG_old, s_GT_old = s_G_old * f_IG, s_G_old * f_GT
434     theta_L = calibrate_theta_L(theta_N, delta, r, A, theta_G, s_IG_old,
        s_GT_old,
435                                 distortionary, N_target)
436
437     ss_old = steady_state_for_policy(theta_N, delta, r, A, theta_G, s_G_old
        , f_IG, f_GT,
438                                      theta_L, distortionary)
439     ss_new = steady_state_for_policy(theta_N, delta, r, A, theta_G, s_G_new
        , f_IG, f_GT,
440                                      theta_L, distortionary)
```

L428–L431 Signature: technology/preferences, θ_G, old/new *total* government-spending ratios (the "ΔG/Y" experiment), composition fractions (held fixed across the experiment), labor-supply target, financing rule, shock duration.

L435 Translates `financing` into the boolean `distortionary`.

L436 Splits the *baseline* total into its two composition levels, for calibrating θ_L.

L437–L438 Calibrates θ_L once, at the baseline policy — used for *both* steady states below.

L440–L443 Builds both steady states, holding (f_{IG}, f_{GT}) and θ_L fixed, varying only the total. Each call internally runs the outer N-fixed-point loop of Section 3 when $\theta_G > 0$.

> **On what the app's "Cumulative Effects" panel compares.** The app's Panel 1 does *not* directly display `ss_old`/`ss_new` from this function. Since `run_experiment` recalibrates θ_L fresh from whatever the *current* sidebar baseline happens to be, its N always lands on exactly `N_target` by construction — which would silently mask any real labor-supply response to composition, financing, θ_G, or r changes (dumping the whole adjustment onto consumption instead). The app therefore computes a separate "current settings" steady state for Panel 1 using `steady_state_for_policy` directly, with theta_L held *fixed* at a benchmark calibration shared with the comparison's left-hand column, so N moves honestly there. `run_experiment`'s own recalibration remains exactly right for its actual purpose here: isolating the effect of the ΔG/Y lever specifically (the app's "Marginal Effects" panel), holding every other structural parameter (including preferences) fixed on both sides of that one comparison.

```
445 dY = ss_new.Y - ss_old.Y
446 dG = ss_new.G - ss_old.G
447 multiplier_long_run = dY / dG if abs(dG) > 1e-12 else float("nan")
```

L445–L447 The long-run output multiplier, $\Delta Y/\Delta G$, an *exact* finite difference of two closed-form steady states — not affected by the log-linear machinery at all, and therefore well defined even when the transition path below fails to exist (see the saddle-path discussion in Section 6).

```
449 if permanent:
450     ss_ref = ss_new
451     k0 = np.log(ss_old.K / ss_new.K)
452     kg0 = np.log(ss_old.KG / ss_new.KG) if ss_new.KG > 0 else 0.0
453     g_path = np.zeros(1)
454 else:
455     ss_ref = ss_old
456     k0 = 0.0
457     kg0 = 0.0
458     dG_frac = (ss_new.G - ss_old.G) / ss_old.G if ss_old.G != 0 else 0.0
459     g_level = np.log(1.0 + dG_frac)
460     g_path = np.concatenate([np.full(duration_years, g_level), np.zeros(1)
        ])
```

L449 **Permanent:** linearize around ss_ref = ss_new.

L450 $\hat{k}_0 = \log(K_{\text{old}}/K_{\text{new}})$ — capital is predetermined, still at its old level.

L451 Same logic for public capital, guarded against $K^G_{\text{new}} = 0$.

L452 No additional shock beyond what's baked into `ss_ref`.

L454 **Temporary:** linearize around the common ss_ref = ss_old.

L455–L456 Both initial gaps are 0 — since only G/Y changed here (composition, financing, θ_G all fixed across old and new), capital starts exactly where the pre-shock steady state already had it.

L457–L460 A finite-duration government-spending shock path: $g_{\text{level}} = \log(1 + \Delta G/G_{\text{old}})$, repeated for `duration_years` periods, then implicitly zero (reverting to ss_ref = ss_old) for the remainder of the simulated horizon. This is the correctly-posed way to model a temporary shock discussed in Section 6: the forcing genuinely returns to exactly 0 — i.e. to `ss_ref` itself — so the saddle-path solver's implicit "no more forcing beyond the horizon" assumption is satisfied.

```
462 path = simulate_transition(ss_ref, g_path, k0=k0, kg0=kg0, T_sim=T_sim)
463
464 Y_level_0 = ss_ref.Y * float(np.exp(path.Y[0] / 100.0))
465 dY0 = Y_level_0 - ss_old.Y
466 dG0 = ss_new.G - ss_old.G
467 multiplier_impact = dY0 / dG0 if abs(dG0) > 1e-12 else float("nan")
```

L462 Calls Section 7's function.

L464 Converts the impact-period deviation back into a level.

L465–L467 The impact multiplier, always relative to the original steady state.

```
469 def shift(field_ref, X_ref, X_old):
470     return field_ref + 100.0 * np.log(X_ref / X_old)
471
472 path.Y = shift(path.Y, ss_ref.Y, ss_old.Y)
473 path.C = shift(path.C, ss_ref.C, ss_old.C)
474 path.I = shift(path.I, ss_ref.I, ss_old.I)
475 path.K = shift(path.K, ss_ref.K, ss_old.K)
476 if ss_ref.KG > 0 and ss_old.KG > 0:
477     path.KG = shift(path.KG, ss_ref.KG, ss_old.KG)
478 else:
479     path.KG = np.zeros_like(path.KG)
480 path.N = shift(path.N, ss_ref.N, ss_old.N)
481 path.W = shift(path.W, ss_ref.w, ss_old.w)
482 if permanent:
483     path.G = np.full_like(path.G, 100.0 * np.log(ss_new.G / ss_old.G))
484 else:
485     path.G = shift(path.G, ss_ref.G, ss_old.G)
```

L469–L470 Re-expresses a path from % deviation from `ss_ref` into % deviation from `ss_old`.

L472–L475 Applied to Y, C, I, K, so every chart is measured relative to the *original* steady state.

L476–L479 Same shift for K^G, guarded against $\log(0/0)$.

L480–L481 Same shift for N, w.

L482–L485 G: constant line under a permanent shock; ordinary shift under a temporary one.

```
487 return PolicyExperiment(ss_old=ss_old, ss_new=ss_new, path=path,
488                         multiplier_long_run=multiplier_long_run,
489                         multiplier_impact=multiplier_impact)
```

L487–L489 Assembles and returns the full `PolicyExperiment`.

Reference

Baxter, Marianne, and Robert G. King. 1993. "Fiscal Policy in General Equilibrium." *American Economic Review* 83 (3): 315–334.